



Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Random Forests

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Trees that agree do not average away much

The variance of an average of B numbers with correlation rho

$$\text{rho} \times \text{sigma}^2 + (1 - \text{rho}) \times \text{sigma}^2 / B$$

rho = 1

Trees agree perfectly: averaging removes nothing, however many there are.

rho = 0

Trees share nothing: a large enough average removes all of their noise.

0.2581

tree correlation, bagging

100 of 100

trees asking PAY_0 first

A random subset of columns at every split

4 of 19

columns tried at each split ("sqrt")

0.2105

chance a given column is tried

PAY_0 comes first in 58 of 300 trees (19.3 percent), about as often as it is in the root's draw at all. The first questions use 13 different columns.

First question	Trees
PAY_0	58
PAY_2	56
PAY_3	40
PAY_4	37
PAY_5	30
PAY_6	28
LIMIT_BAL	16

A column a split cannot see cannot be chosen there: the trees differ from the first question.

The forest beside the bagged model: full trees

Full trees	Validation ROC AUC	Average precision	Tree correlation
Bagging, 100	0.7602	0.5105	0.2581
Random forest, 300	0.7600	0.5146	0.2487

With full trees, the random columns change the first questions and barely change the agreement or the ranking.

Full trees already disagree on many accounts, so there is little agreement left to remove.

The name on today's class

Leo Breiman, October 2001

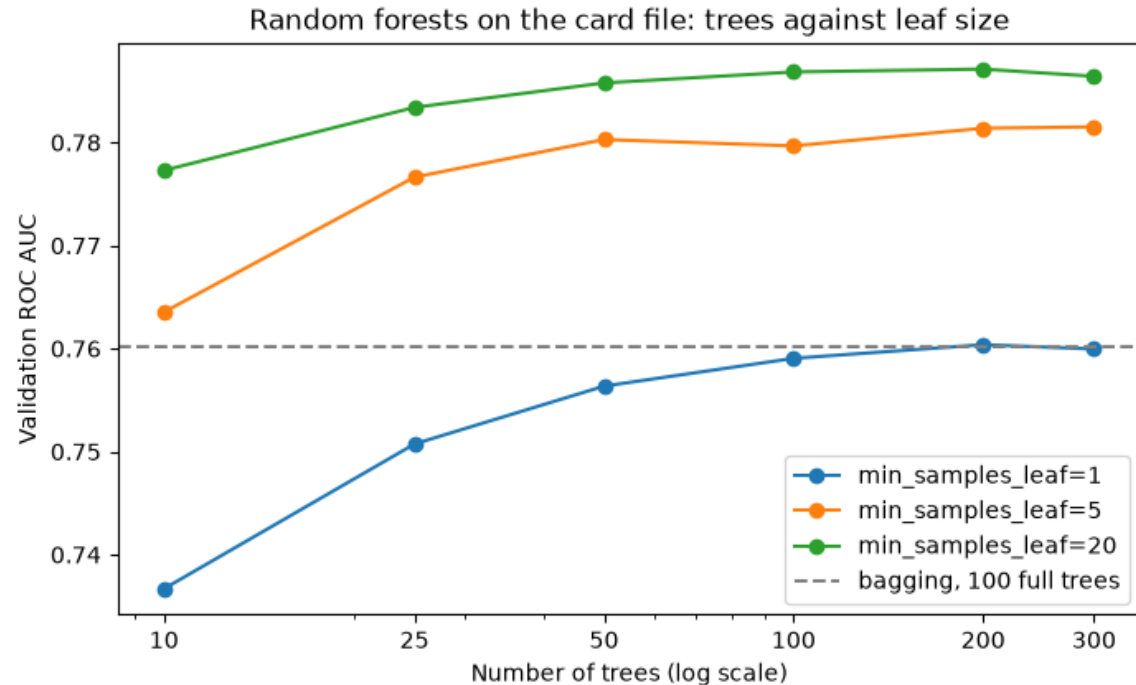
"Random Forests"

Machine Learning, volume 45, issue 1, pages 5 to 32

The same author as week 1's 1996 paper; the title is scikit-learn's class name.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1023/A:1010933404324). Bibliographic facts only; the paper itself was not read for this course.

Leaf size matters more than tree count here



Min leaf	One tree	Correlation	300 trees
1	0.6144	0.2487	0.7600
5	0.6801	0.4116	0.7815
20	0.7257	0.6311	0.7864

Bigger leaves: the trees agree more, each tree alone is much better, and the forest improves.

An average is as good as its members and as independent as they are. On this file, better members win.

max_features=None is bagging

100 full trees	Columns per split	Validation ROC AUC	Tree correlation
RandomForestClassifier, max_features=None	19	0.7595	0.2577
BaggingClassifier	19	0.7602	0.2581

Equal settings, nearly equal numbers: every tree asks PAY_0 first. Today's AI check.

Class weights: the same ranking, a higher level

2.2602

weight of each default

0.6420

weight of each other account

300 trees, 5-row leaves	Validation ROC AUC	Mean output
no weights	0.7815	0.2223
balanced	0.7806	0.3735
balanced_subsample	0.7795	0.3217

Validation default rate 0.2212. A weighted forest's output is the model's score, not a probability.

Choosing a leaf size on the training rows

min_samples_leaf	Mean of 5 folds	SD	Rank
5	0.7709	0.0045	6
10	0.7767	0.0044	5
20	0.7800	0.0039	4
40	0.7827	0.0025	1
80	0.7826	0.0028	2
160	0.7820	0.0023	3

40

rows per leaf, chosen

0.7844

validation ROC AUC,
reported once

50-tree forests, five folds of the training rows. Fold mean of the chosen leaf size: 0.7827, the best of six noisy numbers.

The validation rows look and report. They never choose.

Excel side by side

Excel side by side (tested in Excel)

`=INT(SQRT(19))`

`=INT(0.5*19)`

`=1-COMBIN(18,4)/COMBIN(19,4)`

`=CORREL(B2:B6001,C2:C6001)`

ten CORREL pairs, then `=AVERAGE(H1:H10)`

`=ROWS(A2:A18001)/(2*COUNTIF(A2:A18001,1))`

`=AVERAGE(C1:C5)` and `=STDEV.P(C1:C5)`

4 columns per split, "sqrt"

9 columns per split, `max_features=0.5`

0.2105, a given column tried

bagging 0.2564, forest 0.2362

0.2455, five trees

2.2602, weight of a default

the search's mean and SD

No forest and no tree in Excel. The arithmetic around one, typed into Excel by script and checked against Python.

Save the day: one file's history

```
git log --oneline -- results.csv
```

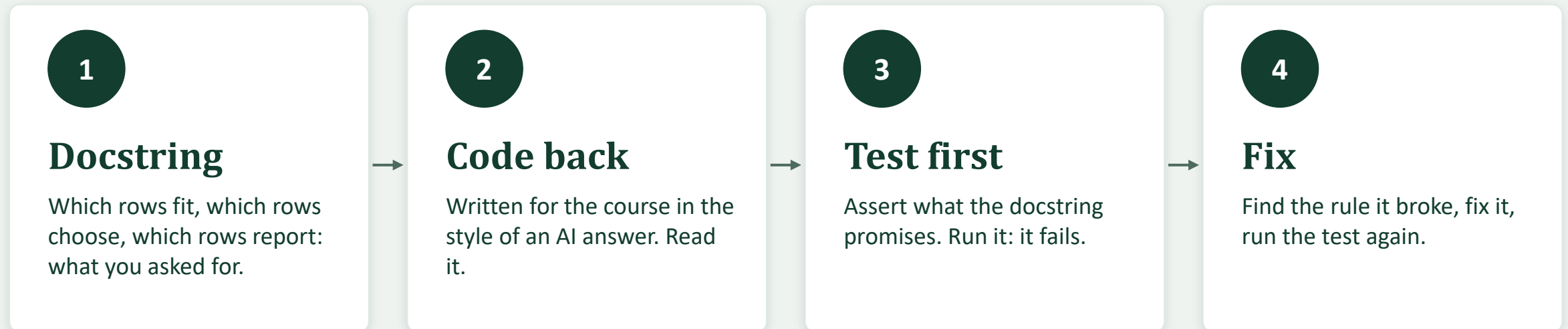
```
Experiment log: leaf sizes, class weights, the searched forest, extra trees  
Experiment log: bagging and a random forest of full trees, validation rows
```

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1  
bagging_100_full_trees,validation,0.7602,0.5105,0.5000,0.8143,0.6357,0.3760,0.4725  
random_forest_300_full_trees,validation,0.7600,0.5146,0.5000,0.8153,0.6402,0.3768,0.4744  
random_forest_300_leaf_5,validation,0.7815,0.5448,0.5000,0.8175,0.6643,0.3534,0.4614  
random_forest_300_leaf_20,validation,0.7864,0.5552,0.5000,0.8185,0.6750,0.3459,0.4574  
random_forest_300_leaf_5_balanced,validation,0.7806,0.5386,0.5000,0.7818,0.5060,0.5750,0.5383  
random_forest_50_leaf_40_searched,validation,0.7844,0.5534,0.5000,0.8198,0.6814,0.3482,0.4608  
extra_trees_300_leaf_5,validation,0.7814,0.5490,0.5000,0.8202,0.6782,0.3557,0.4666
```

Two commits touched results.csv; the code commit did not. A path after -- gives that file's history alone.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.



Today: a "random forest" with `max_features=None`: bagging under another class name.

Code that runs is not the same as code that is right.

What you can do now

- Measure how much an ensemble's trees agree
- Work out the columns a split tries
- `max_features=None` is bagging
- Leaf size moves a forest more than tree count
- Class weights: same ranking, the model's score